Lab2：BombLab 实验报告

本报告根据本次个人 `bomb` 和已完成的 `defuser.c` 分析各关。下面只保留与求解有关的汇编和 C 代码；32 位计算使用 `uint32_t`，输出部分由模板中的 `write_answer` 完成。

一、Phase 1：函数调用与计算关系

这一关需要输入一个数。程序把 challenge 的不同部分交给三个小函数计算，再把结果组合成答案。分析时先确定每次调用传入什么、返回值交给谁，再将三个函数的表达式代入即可。

输入要求与调用关系

`phase_01` 要求一个无符号整数，读取后调用 `p1_compute`，将返回值与输入比较：

```
40209d: mov    $0x1,%esi
4020aa: call   402c50 <valid_u32_fields>
...
4020d3: cmp    $0x1,%eax
4020d6: jne    4020b3 <phase_01+0x23>
...
4020d8: mov    %r12,%rdi
4020db: call   402050 <p1_compute>
4020e0: cmp    %eax,-0x14(%rbp)
4020e3: sete   %al
```

因此本关要恢复的就是 `p1_compute(challenge)`。该函数先准备参数，再依次调用三个辅助函数：

```
402055: mov    %rdi,%rax
402058: mov    %rdi,%rdx
40205b: mov    %edi,%esi
40205d: shr    $0x20,%rax
402061: mov    %edi,%r8d
402064: shr    $0x10,%rdx
402068: mov    %eax,%edi
40206d: call   402040 <p1_step2>

402072: mov    %rdx,%rsi
402075: mov    %r8d,%edi
402078: mov    %eax,%ecx
40207a: call   402020 <p1_step0>

40207f: mov    %ecx,%esi
402081: mov    %eax,%edi
402083: call   402030 <p1_step1>
```

进入函数时 `%rdi` 是 challenge。右移和取低 32 位的操作对应三个变量：

```
uint32_t low = (uint32_t)challenge;
uint32_t high = (uint32_t)(challenge >> 32);
uint32_t middle = (uint32_t)(challenge >> 16);
```

第一次调用 `p1_step2(high, low)`，结果保存在 `%ecx`；第二次调用 `p1_step0(low, middle)`，结果留在 `%eax`；最后将这两个结果分别作为第一、第二个参数传给 `p1_step1`。调用关系可以写成：

```
uint32_t s2 = step2(high, low);
uint32_t s0 = step0(low, middle);
uint32_t answer = step1(s0, s2);
```

辅助函数的计算

这三个函数主要使用 `lea`、加法和异或。这里的 `lea 位移(基址,索引,比例),目标` 计算的是“位移 + 基址 + 索引 × 比例”，不读取该地址中的内容。例如 `lea (%rdi,%rdi,2),%eax` 就是计算第一个参数的三倍。

三个函数的核心指令如下：

```
# p1_step2
402044: lea    (%rdi,%rdi,2),%eax
402047: add    %esi,%eax
402049: xor    $0x18fe,%eax

# p1_step0
402024: lea    (%rdi,%rsi,4),%eax
402027: xor    $0x51fa,%eax

# p1_step1
402034: lea    (%rdi,%rdi,2),%eax
402037: add    $0xfbcd,%esi
40203d: xor    %esi,%eax
```

按两个参数 a、b 直接翻译为：

```
uint32_t step2(uint32_t a, uint32_t b) {
    return (3u * a + b) ^ 0x18feu;
}

uint32_t step0(uint32_t a, uint32_t b) {
    return (a + 4u * b) ^ 0x51fau;
}

uint32_t step1(uint32_t a, uint32_t b) {
    return (3u * a) ^ (b + 0xfbcdu);
}
```

C 语言实现

将辅助函数的表达式代回调用关系，得到求解部分：

```
uint32_t low = (uint32_t)challenge;
uint32_t middle = (uint32_t)(challenge >> 16);
uint32_t high = (uint32_t)(challenge >> 32);

uint32_t step2 = (3u * high + low) ^ 0x18feu;
uint32_t step0 = (low + 4u * middle) ^ 0x51fau;
uint32_t value = (3u * step0) ^ (step2 + 0xfbcdu);
return write_answer(out, cap, &value, 1);
```

其中 middle 是 challenge 右移 16 位后的低 32 位，保留的是原数第 16～47 位。它与 low、high 有重叠，不能把三者理解成互不重叠的三段。

二、Phase 2：数组与递推关系

这一关要输入六个数。第一个数直接从 challenge 中取出，后五个数都根据前一个数继续计算。核心是恢复循环里的一轮计算，再确定循环次数和每轮从 challenge 中取哪几位。

初值与循环计算

入口的格式检查和 `sscanf` 返回值检查都要求六个整数。读入数组从 `-0x40(%rbp)` 开始，每项占 4 字节。第一项的检查为：

```
40215a: mov    %r13,%rax
40215d: mov    -0x40(%rbp),%esi
402160: shr    $0x2d,%rax
402164: and    $0x1f,%eax
402167: cmp    %eax,%esi
```

`%r13` 保存 challenge，因此初值为 `(challenge >> 45) & 31`。后续循环从下标 1 开始，右移位数从 14 开始：

```
40216b: mov    $0xe,%ecx
402170: mov    $0x1,%edx
402175: mov    %r13,%rax
402178: lea    (%rsi,%rsi,4),%esi
40217b: shr    %cl,%rax
40217e: sub    %edx,%esi
402180: and    $0x1f,%eax
402183: add    $0xd,%eax
402186: xor    %esi,%eax
402188: mov    -0x40(%rbp,%rdx,4),%esi
40218c: cmp    %eax,%esi
...
402190: add    $0x1,%rdx
402194: add    $0x5,%ecx
402197: cmp    $0x6,%rdx
40219b: jne    402175 <phase_02+0x85>
```

每轮开始时，`%esi` 是前一项，`%rdx` 是下标 i，`%ecx` 是移位位数。前一项先乘 5 再减 i；另一边从 challenge 中取出 5 位，加 13 后与前面的结果异或。这就是当前项的期望值。

比较通过后，输入的当前项留在 `%esi`，成为下一轮的前一项。i 每轮加 1，移位位数每轮加 5，因此 i 为 1 ~ 5，对应移位量为 `9 + 5*i`。

C 语言实现

将循环里的计算直接写入数组：

```
uint32_t values[6];
values[0] = (uint32_t)(challenge >> 45) & 31u;

for (unsigned i = 1; i < 6; ++i) {
    uint32_t field =
        (uint32_t)(challenge >> (9u + 5u * i)) & 31u;
    values[i] = (5u * values[i - 1] - i) ^ (field + 13u);
}

return write_answer(out, cap, values, 6);
```

这里减法要保留无符号运算的行为。例如前一项为 0 时，减 1 会回绕到 `0xffffffff`，后续仍按这个 32 位数继续计算。

三、Phase 3：跳转表与分支计算

这一关的两个输入分别是分支编号和该分支算出的结果。先用 challenge 确定编号和计算参数，再读取跳转表确认每个编号真正对应哪个函数，最后将各分支写成 `switch`。

输入构造与分支选择

`phase_03` 要求两个整数。第一个整数必须等于 challenge 的最低 3 位，第二个整数由 `p3_dispatch` 计算：

```
4022e1: and    $0x7,%edi
4022e4: cmp    %edi,-0x18(%rbp)
...
4022ef: shr    $0x3,%rax
4022f3: shr    $0x20,%rsi
4022f7: xor    %eax,%esi
4022f9: call   402230 <p3_dispatch>
4022fe: cmp    %eax,-0x14(%rbp)
```

调用参数可写为：

```
uint32_t index = (uint32_t)challenge & 7u;
uint32_t x = (uint32_t)((challenge >> 3) ^ (challenge >> 32));
```

`p3_dispatch` 保存 index，并把 x 放到第一个参数位置，随后通过表项跳转：

```
402234: mov    %edi,%eax
402236: mov    %esi,%edi
402238: cmp    $0x7,%eax
40223b: ja     402285 <p3_dispatch+0x55>
40223d: notrack jmp *0x405820(,%rax,8)
```

表项大小为 8 字节。用 `x/8gx 0x405820` 读出地址，并跟随各处的 `jmp`，可知 index 从 0 到 7 时，实际执行的函数依次是 `p3_case_0`、`p3_case_7`、`p3_case_5`、`p3_case_4`、`p3_case_6`、`p3_case_2`、`p3_case_3`、`p3_case_1`。因此输入编号与函数名尾号并不相同。

```
(gdb) x/8gx 0x405820
0x405820:       0x0000000000402258      0x0000000000402260
0x405830:       0x0000000000402268      0x0000000000402270
0x405840:       0x0000000000402278      0x0000000000402280
0x405850:       0x0000000000402248      0x0000000000402250
(gdb) x/gx 0x405828
0x405828:       0x0000000000402260
(gdb) x/i 0x402260
   0x402260 <p3_dispatch+48>:   jmp    0x402220 <p3_case_7>
(gdb) disassemble p3_case_7
Dump of assembler code for function p3_case_7:
   0x0000000000402220 <+0>:     endbr64
   0x0000000000402224 <+4>:     lea    (%rdi,%rdi,1),%eax
   0x0000000000402227 <+7>:     xor    $0xe987,%eax
   0x000000000040222c <+12>:    ret
End of assembler dump.
```

分支运算与 C 语言实现

各分支主要有三种运算形式，分别是乘法后异或、乘法后加常量，以及“异或结果加右移结果”。下面各取一个关键片段：

```
# p3_case_0: 2*x 后异或常量
4021b4: lea    (%rdi,%rdi,1),%eax
4021b7: xor    $0xf5b1,%eax

# p3_case_3: 3*x 加常量
4021e4: lea    0xb8a4(%rdi,%rdi,2),%eax

# p3_case_2: 保留原 x，分别异或和右移，再相加
4021d4: mov    %edi,%eax
4021d6: shr    $1,%edi
4021d8: xor    $0xa3e0,%eax
4021dd: add    %edi,%eax
```

最后一种写法中，`%eax` 保留原来的 x，`%edi` 变成 `x >> 1`，所以得到 `(x ^ 0xa3e0) + (x >> 1)`。其他函数按同样的方法读出比例和常量，再按实际跳转表顺序整理：

```
uint32_t index = (uint32_t)challenge & 7u;
uint32_t x = (uint32_t)((challenge >> 3) ^ (challenge >> 32));
uint32_t values[2] = {index, 0};

switch (index) {
    case 0: values[1] = (2u * x) ^ 0xf5b1u; break;
    case 1: values[1] = (2u * x) ^ 0xe987u; break;
    case 2: values[1] = 5u * x + 0x3c3bbu; break;
    case 3: values[1] = 5u * x + 0xfbbeu; break;
    case 4: values[1] = (x ^ 0x2ef9u) + (x >> 1); break;
    case 5: values[1] = (x ^ 0xa3e0u) + (x >> 1); break;
    case 6: values[1] = 3u * x + 0xb8a4u; break;
    case 7: values[1] = 5u * x + 0x4fba0u; break;
}

return write_answer(out, cap, values, 2);
```

四、Phase 4：递归查找与返回值计算

这一关先在 0～127 中按二分的方法找到目标，再沿递归调用一层层返回，每层都对下一层的结果做一次计算。答案由整条查找路径共同决定，所以既要还原向左、向右的条件，也要保留递归返回后的运算。

参数与递归条件

`phase_04` 读取一个整数，与 `p4_walk` 的返回值比较。调用前的参数准备对应：

```
uint32_t target = (uint32_t)challenge & 127u;
uint32_t salt = (uint32_t)((challenge >> 7) ^ (challenge >> 32));
uint32_t value = phase4_walk(target, 0, 127, salt);
```

四个参数依次为 target、low、high、salt。前三个确定查找目标和区间，salt 是整个递归过程中都要使用的计算参数。`p4_walk` 开头先求中点：

```
402314: mov    %edi,%eax
402316: mov    %edx,%edi
402318: sub    %esi,%edi
40231a: shr    $1,%edi
40231c: add    %esi,%edi
40231e: cmp    %eax,%edi
402320: je     402370 <p4_walk+0x60>
```

这里 `%eax` 保存 target，`%edi` 经过计算变为 `low + ((high-low) >> 1)`，即 middle。命中时跳到的代码为：

```
402370: lea    0x50(%rdi),%eax
402373: xor    %ecx,%eax
```

因此递归结束条件是 `target == middle`，返回值为 `(middle + 0x50) ^ salt`。

分支与返回过程

没有命中时，程序先保存本层的 `middle + salt`，再比较 target 与 middle：

```
402327: lea    (%rdi,%rcx,1),%ebx
40232e: cmp    %edi,%eax
402330: jb     402350 <p4_walk+0x40>
```

`jb` 对应 `target < middle`，进入左半区间并将 high 改为 `middle-1`；否则进入右半区间，将 low 改为 `middle+1`。以左分支为例，关键代码如下：

```
402350: lea    -0x1(%rdi),%edx
402353: mov    %eax,%edi
402355: call   402310 <p4_walk>
40235a: lea    (%rax,%rax,2),%eax
40235d: xor    %ebx,%eax
402364: xor    $0x1b,%eax
```

递归返回后，将下一层结果乘 3，再异或本层保存的 `middle+salt` 和 `0x1b`。右分支做同样的运算，但最后的常量为 `0x14`。这说明最终答案不是找到的 target，而是递归结果经过每层运算后的值。

```
Phase 4 challenge: 0xbacb315bdc413cd5
Input (unsigned decimal integers):
target=85 low=0 high=127 middle=63 salt=0xd73b322
target=85 low=64 high=127 middle=95 salt=0xd73b322
target=85 low=64 high=94 middle=79 salt=0xd73b322
target=85 low=80 high=94 middle=87 salt=0xd73b322
target=85 low=80 high=86 middle=83 salt=0xd73b322
target=85 low=84 high=86 middle=85 salt=0xd73b322
```

```
(gdb) printf "parent middle=%u, child=%u, middle+salt=%u\n", (unsigned int)$ebx - (unsigned int)$ecx
, (unsigned int)$eax, (unsigned int)$ebx
parent middle=83, child=225686407, middle+salt=225686389
(gdb) printf "expected=%u\n", (unsigned int)(3 * (unsigned int)$eax) ^ (unsigned int)$ebx ^ 0x14
expected=623421940
(gdb) x/6i $pc
=> 0x40233c <p4_walk+44>:       lea    (%rax,%rax,2),%eax
   0x40233f <p4_walk+47>:       xor    %ebx,%eax
   0x402341 <p4_walk+49>:       mov    -0x8(%rbp),%rbx
   0x402345 <p4_walk+53>:       leave
   0x402346 <p4_walk+54>:       xor    $0x14,%eax
   0x402349 <p4_walk+57>:       ret
(gdb) tbreak *0x402349
Temporary breakpoint 2 at 0x402349
(gdb) continue
Continuing.

Temporary breakpoint 2, 0x0000000000402349 in p4_walk ()
(gdb) printf "actual=%u\n", (unsigned int)$eax
actual=623421940
```

C 语言实现

将结束条件和左右分支合并即可。这里用 child 表示下一层的返回值，当前 `defuser.c` 则直接将递归调用写在表达式里。

```
static uint32_t phase4_walk(uint32_t target, uint32_t low,
                            uint32_t high, uint32_t salt) {
    uint32_t middle = low + ((high - low) >> 1);
    if (target == middle) {
        return (middle + 0x50u) ^ salt;
    }
    if (target < middle) {
        uint32_t child = phase4_walk(target, low, middle - 1u, salt);
        return (3u * child) ^ (middle + salt) ^ 0x1bu;
    }
    uint32_t child = phase4_walk(target, middle + 1u, high, salt);
    return (3u * child) ^ (middle + salt) ^ 0x14u;
}
```

求解函数按前面恢复的参数调用它，并输出返回值。例如 target 为 85 时，中点依次经过 `63` → `95` → `79` → `87` → `83` → `85`，从 85 命中处开始计算，再按相反顺序逐层返回。

五、Phase 5：链表布局与遍历

这一关根据 challenge 选择链表起点和访问次数，边走边更新一个数，最后输出计算结果和最后一个节点的 tag。分析重点是读出节点各字段的含义、实际连接顺序，以及循环究竟处理了多少个节点。

节点布局与连接关系

`phase_05` 要求两个整数。`p5_traverse` 的返回值对应第一个数，另一个结果通过传入的指针写回，对应第二个数。函数中对节点的主要访问为：

```
402429: mov    0x4(%rdx),%esi
402433: add    (%rdx),%eax
402447: mov    0x8(%rdx),%rdx
```

当前节点由 `%rdx` 指向。偏移 0 和 4 处各读取一个 32 位整数，分别参与加法和异或；偏移 8 处读取一个 64 位地址，并将它作为下一节点。因此节点结构对应：

```
struct Node {
    uint32_t value;
    uint32_t tag;
    struct Node *next;
};
```

用 GDB 读取 `p5_nodes`，以 `0x405ba0` 为节点 0，每增加 16 字节编号加 1。例如首节点的 value 为 `0x0414`，tag 为 `0x0985`，next 为 `0x405c50`，也就是节点 11。继续沿指针读取，整条链为：

```
0 → 11 → 7 → 6 → 3 → 8 → 2 → 4 → 5 → 10 → 1 → 9 → NULL
```

起点由 `0x405b60` 处的头指针表决定：

```
40240d: and    $0x7,%eax
402414: mov    0x405b60(,%rax,8),%rdx
```

读取该表后，challenge 最低三位取 0～7 时，起点依次是节点 `0`、`11`、`7`、`6`、`3`、`8`、`2`、`4`。因此头表下标、节点编号和节点在链上的位置是三件不同的事。

```
(gdb) x/48wx 0x405ba0
0x405ba0 <p5_nodes>:    0x00000414      0x00000985      0x00405c50      0x00000000
0x405bb0 <p5_nodes+16>: 0x00008a2c      0x0000085f      0x00405c30      0x00000000
0x405bc0 <p5_nodes+32>: 0x00004b05      0x0000075d      0x00405be0      0x00000000
0x405bd0 <p5_nodes+48>: 0x00000557      0x00000633      0x00405c20      0x00000000
0x405be0 <p5_nodes+64>: 0x0000d2a5      0x0000021e      0x00405bf0      0x00000000
0x405bf0 <p5_nodes+80>: 0x0000c8e9      0x00000462      0x00405c40      0x00000000
0x405c00 <p5_nodes+96>: 0x0000a905      0x00000566      0x00405bd0      0x00000000
0x405c10 <p5_nodes+112>:        0x0000491a      0x00000a0d      0x00405c00      0x00000000
0x405c20 <p5_nodes+128>:        0x0000685e      0x00000b37      0x00405bc0      0x00000000
0x405c30 <p5_nodes+144>:        0x00005ba9      0x0000011b      0x00000000      0x00000000
0x405c40 <p5_nodes+160>:        0x00004dec      0x00000022      0x00405bb0      0x00000000
0x405c50 <p5_nodes+176>:        0x00005835      0x00000334      0x00405c10      0x00000000
(gdb) x/gx 0x405ba8
0x405ba8 <p5_nodes+8>:  0x0000000000405c50
(gdb) p/d (0x405c50 - 0x405ba0) / 16
$1 = 11
(gdb) x/4wx 0x405c50
0x405c50 <p5_nodes+176>:        0x00005835      0x00000334      0x00405c10      0x00000000
```

更新规则与访问次数

累加值的初值来自 challenge 的移位和异或：

```
uint32_t acc =
    (uint32_t)((challenge >> 6) ^ (challenge >> 32)) ^ 0x33f7u;
```

每个节点的计算由下面几条指令完成：

```
402433: add    (%rdx),%eax
402435: lea    (%rax,%rax,4),%eax
402438: mov    %esi,(%r8)
40243b: xor    %esi,%eax
```

`%eax` 是 acc，`%esi` 是当前 tag，`%r8` 是第二个结果的输出地址。于是更新关系为 `acc = 5*(acc+value) ^ tag`，同时将当前 tag 写回。由于后续节点会覆盖前一次写入，最后留下的自然是最后访问节点的 tag。

程序处理完首节点后，才检查 `(challenge >> 3) & 3` 是否为 0；为 0 就直接返回，否则沿 next 继续处理。所以这一数值表示还要继续访问的节点数，总次数应为 `((challenge >> 3) & 3) + 1`，范围为 1 ~ 4。

C 语言实现

在 `defuser.c` 中将固定节点数据保存到 nodes 数组，用节点编号表示 next，遍历过程与使用指针相同。关键代码为：

```
static const unsigned heads[8] = {0, 11, 7, 6, 3, 8, 2, 4};
unsigned node = heads[challenge & 7u];
unsigned count = (unsigned)((challenge >> 3) & 3u) + 1u;
uint32_t values[2] = {
    (uint32_t)((challenge >> 6) ^ (challenge >> 32)) ^ 0x33f7u,
    0
};

for (unsigned i = 0; i < count; ++i) {
    values[0] = (5u * (values[0] + nodes[node].value))
                ^ nodes[node].tag;
    values[1] = nodes[node].tag;
    node = nodes[node].next;
}

return write_answer(out, cap, values, 2);
```

例如从头表第 4 项开始、访问 3 次时，实际节点为 `3 → 8 → 2`，第二个输出应是节点 2 的 tag，即 `0x075d = 1885`。

六、Phase 6：函数指针表与两次调用

这一关先从函数表中选一个运算处理 x，再把结果交给表中的另一个运算。两个函数由 challenge 的不同位决定，中间和最后还各有一次固定运算。恢复函数表的顺序和各函数的表达式，就能写出完整计算。

调用关系与函数表

`p6_dispatch` 先构造参数 `x = (uint32_t)((challenge >> 6) ^ (challenge >> 32))`，再执行两次间接调用：

```
402569: mov    %rbx,%rax
402570: and    $0x38,%ebx
402573: and    $0x7,%eax
402576: call   *0x405e00(,%rax,8)
40257d: xor    $0xd25b,%eax
402582: mov    %eax,%edi
402584: call   *0x405e00(%rbx)
40258f: add    $0xd59b,%eax
```

第一次的表下标是 `challenge & 7`。第二次 `%rbx` 中的 `challenge & 0x38` 已经是字节偏移，除以表项大小 8，得到下标 `(challenge >> 3) & 7`。

两次调用使用同一张表。读取 `p6_ops` 可知，表下标 0 ~ 7 依次对应函数 `op3`、`op1`、`op0`、`op2`、`op5`、`op0`、`op4`、`op3`。第一次的结果先异或 `0xd25b` 再传给第二次，第二次返回后加 `0xd59b`，得到唯一的输出整数。

运算函数与 C 语言实现

各函数中的普通运算可以按寄存器里的值直接翻译。以 op1 为例：

```
4024e4: lea    0x0(,%rdi,8),%edx
4024eb: lea    0x49be(%rdx),%eax
4024f1: sub    %edi,%edx
4024f3: xor    %edx,%eax
```

`%edx` 先为 `8*x`，随后减去原来的 x 变成 `7*x`；另一边 `%eax` 为 `8*x+0x49be`。因此返回 `(8*x+0x49be) ^ (7*x)`。

需要另外注意 op4 和 op5 使用的是循环移位：

```
# op4
402536: rol    $0x5,%eax
402539: xor    $0x3a06,%eax

# op5
402546: ror    $0x2,%eax
```

`rol` 将左侧移出的位补回右侧，`ror` 则反过来。由于这里操作的是 32 位数，分别写成 `(x<<5)|(x>>27)` 和 `(x>>2)|(x<<30)`。按同样方式整理六个函数：

```
static uint32_t phase6_op0(uint32_t x) { return (x ^ 0xd231u) + 4u * x; }
static uint32_t phase6_op1(uint32_t x) { return (8u * x + 0x49beu) ^ (7u * x); }
static uint32_t phase6_op2(uint32_t x) { return (x ^ (x >> 5)) + 0x9868u; }
static uint32_t phase6_op3(uint32_t x) { return (x + 0x13dbu) ^ (x >> 1); }
static uint32_t phase6_op4(uint32_t x) { return ((x << 5) | (x >> 27)) ^ 0x3a06u; }
static uint32_t phase6_op5(uint32_t x) { return (x >> 2) | (x << 30); }
```

再按原表顺序组织调用：

```
static uint32_t (*const operations[8])(uint32_t) = {
    phase6_op3, phase6_op1, phase6_op0, phase6_op2,
    phase6_op5, phase6_op0, phase6_op4, phase6_op3
};

uint32_t x = (uint32_t)((challenge >> 6) ^ (challenge >> 32));
x = operations[challenge & 7u](x) ^ 0xd25bu;
uint32_t value = operations[(challenge >> 3) & 7u](x) + 0xd59bu;
return write_answer(out, cap, &value, 1);
```

七、Honor：前缀构造与三个整数的计算

Honor 分成两部分：先让输入中的指定位置出现 `OPEN`，通过入口检查；再根据本关 challenge 和前六关答案生成的 context，算出后面的三个整数。分析时分别确定字符串各部分的位置和整数计算公式，最后将它们拼成第七行口令。

入口检查与口令结构

完整通过前六关后，通过管道或文件提供第七行即可进入 Honor 检查。驱动已经根据前六行计算 context，并将它与 Honor challenge 一起传给 `solve_secret`。

`secret_gate` 中的关键位置为：

```
40271b: lea    -0xd0(%rbp),%r12
...
402763: mov    %r12,%rdi
402766: call   4026e0 <secret_copy>
40276b: cmpl   $0x4e45504f,-0xb0(%rbp)
402775: jne    40274b <secret_gate+0x4b>
...
40277d: lea    -0xac(%rbp),%rdi
402784: call   402620 <secret_phase>
```

复制从 `rbp-0xd0` 开始，检查字段位于 `rbp-0xb0`，相差 32 字节；随后从 `rbp-0xac` 开始解析整数，距复制起点 36 字节。因此输入应由“32 字节占位、4 字节检查内容、三个整数”组成。

`secret_copy` 的复制循环如下：

```
4026f0: movzbl (%rsi,%rax,1),%edx
4026f4: mov    %dl,(%rdi,%rax,1)
4026f7: add    $0x1,%rax
4026fb: test   %dl,%dl
4026fd: jne    4026f0 <secret_copy+0x10>
```

它遇到字符串末尾的 `\0` 才停止，不会在 32 字节处停下，因此输入可以继续覆盖到相邻的检查字段。小端序下，`0x4e45504f` 在内存中的字节为 `4f 50 45 4e`，对应字符 `OPEN`。前面用 32 个 `A` 占位即可。

复制前还有长度条件：`strlen(input)-36` 按无符号数比较必须不超过 127，即长度在 36 ~ 163 字节之间。由此确定的口令结构为：

```
32 个 A + OPEN + mode + 空格 + answer + 空格 + check
```

这里处理的是局部对象中相邻的字段，后续仍由程序正常调用 `secret_phase`。

```
Breakpoint 1, 0x0000000000402700 in secret_gate ()
(gdb) p/x $rsi
$1 = 0xab5368d172d4fa4d
(gdb) p/x $rdx
$2 = 0x7976794a66b73694
(gdb) x/i 0x40271b
   0x40271b <secret_gate+27>:   lea    -0xd0(%rbp),%r12
(gdb) x/8i 0x402763
   0x402763 <secret_gate+99>:   mov    %r12,%rdi
   0x402766 <secret_gate+102>:  call   0x4026e0 <secret_copy>
   0x40276b <secret_gate+107>:  cmpl   $0x4e45504f,-0xb0(%rbp)
   0x402775 <secret_gate+117>:  jne    0x40274b <secret_gate+75>
   0x402777 <secret_gate+119>:  mov    %r14,%rdx
   0x40277a <secret_gate+122>:  mov    %r13,%rsi
   0x40277d <secret_gate+125>:  lea    -0xac(%rbp),%rdi
   0x402784 <secret_gate+132>:  call   0x402620 <secret_phase>
```

```
Breakpoint 2, 0x000000000040276b in secret_gate ()
(gdb) p/d ($rbp-0xb0)-($rbp-0xd0)
$3 = 32
(gdb) p/d ($rbp-0xac)-($rbp-0xd0)
$4 = 36
(gdb) x/4cb $rbp-0xb0
0x7fffffffd5d0: 79 'O'  80 'P'  69 'E'  78 'N'
(gdb) x/wx $rbp-0xb0
0x7fffffffd5d0: 0x4e45504f
(gdb) x/s $rbp-0xac
0x7fffffffd5d4: "0 4123460129 2034409213"
```

三个整数的计算关系

`secret_phase` 要求三个无符号整数，并将 challenge、context 分别保存在 `%r12`、`%r13`。第一个整数与 `context & 3` 比较，因此令 `mode = context & 3`。

接下来分别将 challenge 和 context 的高低 32 位异或：

```
40269a: shr    $0x20,%rsi
40269e: shr    $0x20,%r8
4026a2: xor    %r12d,%esi
4026a5: xor    %r13d,%r8d
...
4026ac: mov    %esi,%edi
4026b1: xor    %r8d,%edi
4026b4: call   402600 <secret_mix>
```

将两次合并的结果分别记为 folded_challenge 和 folded_context，它们再异或后作为 `secret_mix` 的参数：

```
uint32_t folded_challenge = (uint32_t)(challenge ^ (challenge >> 32));
uint32_t folded_context = (uint32_t)(context ^ (context >> 32));
uint32_t x = folded_challenge ^ folded_context;
```

`secret_mix` 先循环左移一位，再加常量并异或：

```
402606: rol    $1,%eax
402608: add    $0x2ca1,%eax
40260d: xor    $0x83b0,%eax
```

所以它的结果为 `mixed = (rol32(x,1) + 0x2ca1) ^ 0x83b0`。调用者还计算了 `mode * 0x2ca1`，并与 context 右移 16 位后的低 32 位异或，最后将这部分与 mixed 合并：

```
402690: imul   $0x2ca1,%ecx,%r9d
...
4026a8: shr    $0x10,%r13
4026ae: xor    %r13d,%r9d
...
4026b9: xor    %r9d,%eax
4026bc: cmp    %eax,-0x28(%rbp)
```

第二个整数因此是 `answer = mixed ^ (mode * 0x2ca1) ^ (uint32_t)(context >> 16)`。

第三个整数的检查为：

```
4026c1: shr    $1,%esi
4026c3: lea    0x83b0(%r8,%rcx,1),%edx
4026cb: add    %esi,%edx
4026cd: xor    %edx,%eax
4026cf: cmp    -0x24(%rbp),%eax
```

这里 `%esi` 是 folded_challenge，`%r8d` 是 folded_context，`%ecx` 是 mode，`%eax` 是 answer。先求 `folded_context + mode + 0x83b0 + (folded_challenge >> 1)`，再与 answer 异或，就得到 check。加法结果按 32 位截断后再参与异或。

C 语言实现

将计算关系和前缀拼接合并，得到 `solve_secret` 的主要代码：

```
uint32_t mode = (uint32_t)context & 3u;
uint32_t folded_challenge = (uint32_t)(challenge ^ (challenge >> 32));
uint32_t folded_context = (uint32_t)(context ^ (context >> 32));
uint32_t x = folded_challenge ^ folded_context;

uint32_t mixed = (((x << 1) | (x >> 31)) + 0x2ca1u) ^ 0x83b0u;
uint32_t answer = mixed ^ (mode * 0x2ca1u)
                      ^ (uint32_t)(context >> 16);
uint32_t check = answer ^ (folded_context + mode + 0x83b0u
                           + (folded_challenge >> 1));

int size = snprintf(out, cap,
                    "AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAOPEN"
                    "%" PRIu32 " %" PRIu32 " %" PRIu32,
                    mode, answer, check);
return size < 0 || (size_t)size >= cap;
```

`OPEN` 后直接接 mode，因为 `secret_phase` 从偏移 36 开始读取，已经跳过了前缀。整个输出不包含换行，换行由驱动添加。

在 seed `20261025` 下，GDB 读取到 Honor challenge 为 `0xab5368d172d4fa4d`，context 为 `0x7976794a66b73694`。复制完成后，偏移 32 处为 `OPEN`，偏移 36 处为 `0 4123460129 2034409213`，与分析一致。

八、运行结果

2026 年 10 月 25 日的公开测试结果为正式六关 `60/60`，Honor `10/10`。

```
Public tests: 60/60
Secret seed 0: PASS
Secret seed 1: PASS
Secret seed 7: PASS
Secret seed 42: PASS
Secret seed 99: PASS
Secret seed 123: PASS
Secret seed 2026: PASS
Secret seed 65535: PASS
Secret seed 987654321: PASS
Secret seed 18446744073709551615: PASS
Secret public tests: 10/10
```

另选不在 `tests/public.json` 中的 seed `20261025`，六关及 Honor 均通过检查。前六行口令依次为：

```
1731235154
30 178 857 4263 21279 106372
6 2362697792
2696985568
4075795618 1885
2134570286
```

Honor 的第七行为：

```
AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAOPEN0 4123460129 2034409213
```

复现命令：

```
{
    ./defuser --seed 20261025
    ./defuser --secret --seed 20261025
} | ./bomb --seed 20261025
```

```
will@Mention:/mnt/d/Desktop/ICS/lab2/ics-2026-bomblab$ ./bomb --seed 20261025 < /tmp/bomblab-all.txt | cat
Phase 1 challenge: 0x8eba8e894f47fa1f
Input (unsigned decimal integers):
Phase 1 defused.
Phase 2 challenge: 0xc237c29670a6a851
Input (unsigned decimal integers):
Phase 2 defused.
Phase 3 challenge: 0x6951131768bbb11e
Input (unsigned decimal integers):
Phase 3 defused.
Phase 4 challenge: 0xbacb315bdc413cd5
Input (unsigned decimal integers):
Phase 4 defused.
Phase 5 challenge: 0x94613bc3fa834674
Input (unsigned decimal integers):
Phase 5 defused.
Phase 6 challenge: 0xd70f945f731b54ba
Input (unsigned decimal integers):
Phase 6 defused.
Congratulations! You've defused the bomb.
Secret phase defused.
```

```
[1 已解除] [2 已解除] [3 已解除] [4 已解除] [5 已解除] [6 已解除]

Congratulations! You've defused the bomb.
[SAFE] 六项检查全部通过。新存档已启用。

你们走回街上。送货司机已经发动了车：
‘昨天耽误了。今天得早点走。’
维修铺的灯装好了，老板正把那张写着‘明天装上’的纸条揭下来。
林夏站在橱窗前看了一会儿，跟上了你。

Honor · 观测台旧备份

林夏准备关闭档案列表，忽然指着最下面一项：‘这里还有个备份。’
文件夹叫‘留给后来的人’，署名是 sky 和许遥。
扉页写着：‘交接完成后，如果你还有时间，可以到观测台坐坐。’
这份旧档案不参与城市的运行。你们沿着楼梯登上观测台，
桌上的阅读器仍在等待旧程序认可的输入。
城市已经安全了。是否继续读下去，由你们自己决定。

Secret phase defused.
[SAFE] 城市照常运行。附加档案已恢复。
```

九、参考资料与工具说明

使用gpt查询汇编语言含义，objdump、GDB用法以及题目实现思路讨论。使用 objdump、GDB 读取汇编、数据表及运行中的值，使用 GCC、Python 编译和检查结果。使用 Codex 辅助核对已有代码、整理推导思路和撰写报告。